Implementation of a Multi-Channel Logic Analyzer for Embedded System Debugging

Aim: To study and understand the working principle of a Logic Analyzer and to learn how to use it for analysing and debugging digital signals and communication protocols such as I2C, SPI, and UART in embedded systems applications.

Components Required:

Sno	Name of the component	Quantity
1	ESP32C6 Dev Module	1
2	8 channel logic analyzer	1
3	Jumper Wires	As Needed

Description of the project: A Logic Analyzer is a digital debugging tool used to capture and analyse digital signals (0s and 1s) in electronic circuits. It is mainly used to debug communication protocols like I2C, SPI, and UART in microcontroller projects. Unlike an oscilloscope, it focuses only on digital timing and logic states. It helps easily to identify errors in embedded systems and digital electronics circuits.

How to Connect and use the logic analyser:

Step 1: Install Pulse View Software

- Download Pulse View (Sigrok software) from the official website.
- Install it on your PC/Laptop.
- After installation, open the Pulse View application.

Step 2: Connect the Logic Analyzer to PC/Laptop

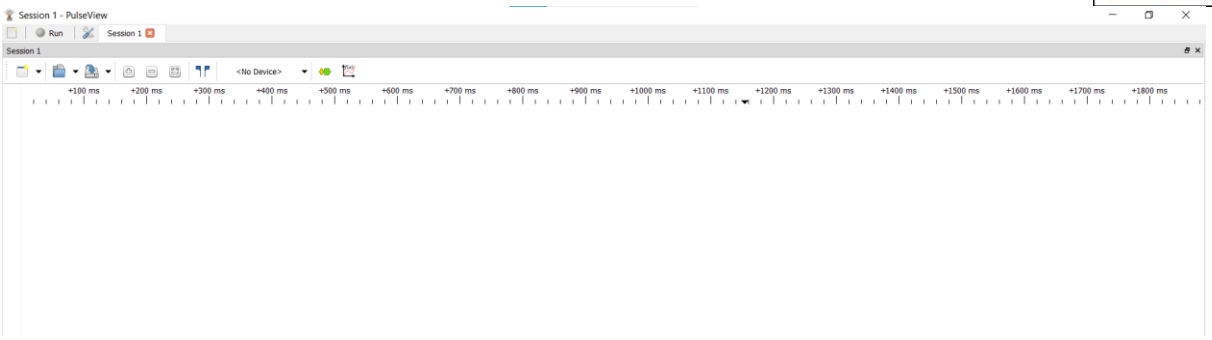
- Download Saleae from the official website
- Install it on your PC/Laptop.

Step 3: Connect the Logic Analyzer to PC/Laptop

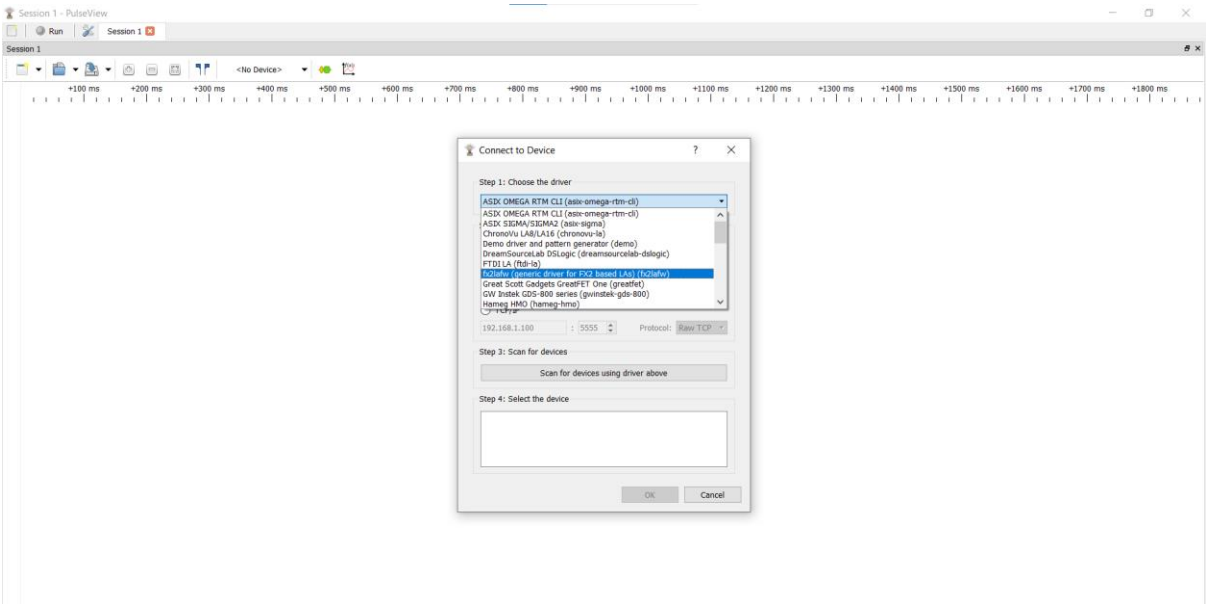
- Connect the logic analyser to your PC/Laptop using a USB cable.
- Most USB logic analysers are plug-and-play.
- Wait for the system to detect the port automatically.

Step 4: Select the Logic Analyzer Device

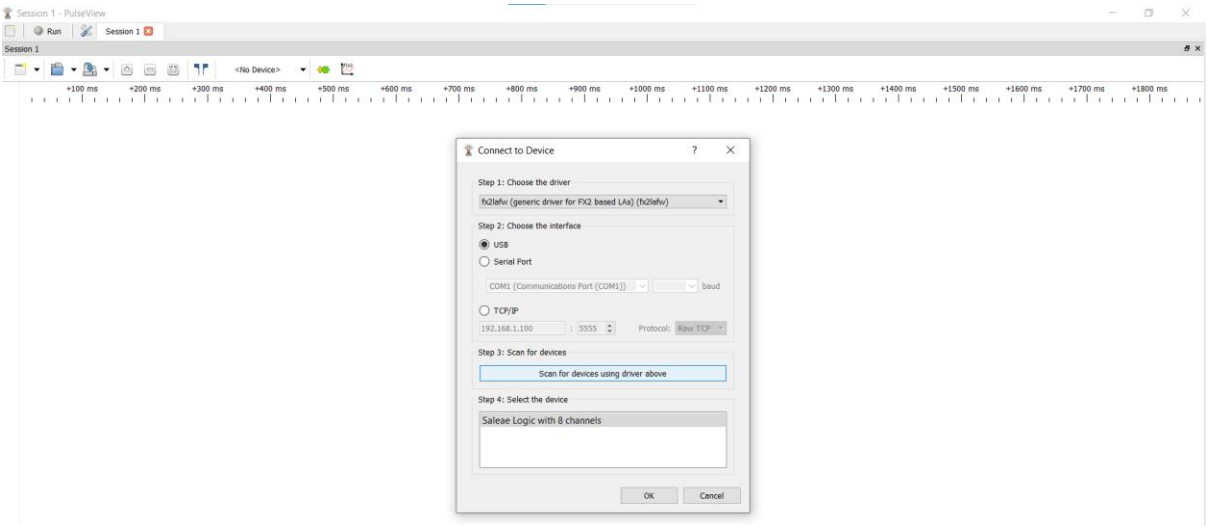
- Open Pulse View.
- Click on the “No Device” option.



- Select your logic analyser (e.g., fx2lafw (generic driver for FX2 based LAs).



- Click “scan for devices using driver above”



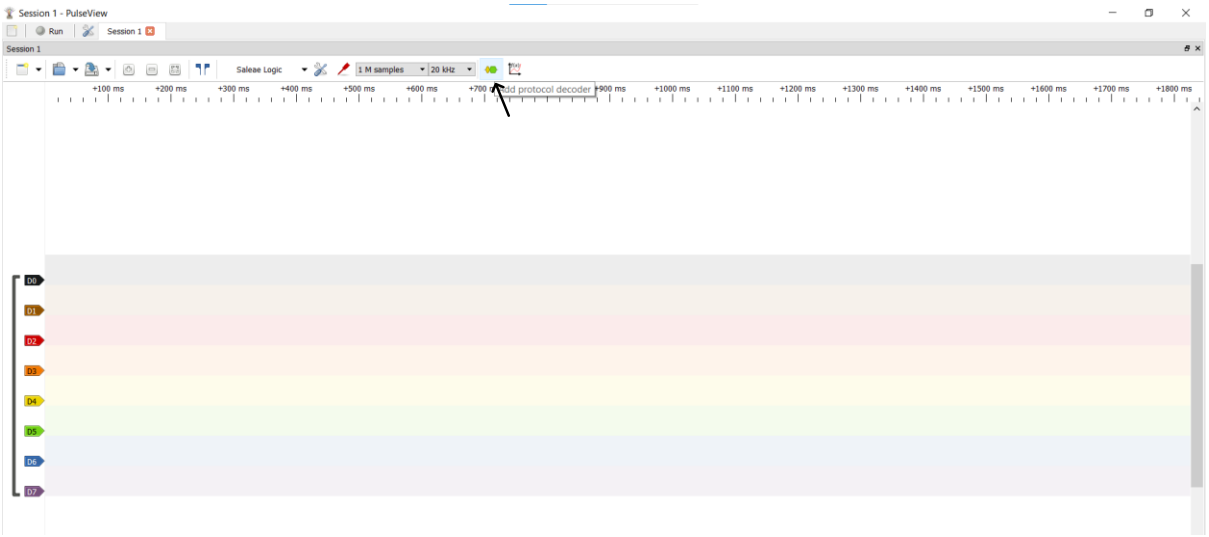
- Now you can see “saleae Logic with 8 channels” Click that and give Ok
- If not detected, check USB connection or driver installation.

Step 5: Connect Probes to the Circuit

- Connect the GND (ground) of the logic analyser to the circuit ground.
- Connect channel pins (CH0, CH1, etc.) to the digital signals you want to monitor (e.g., TX, RX, SCL, SDA, MOSI, MISO, CLK).

Step 6: Configure Settings in Pulse View

- Set the sampling rate (e.g., 1 MHz, depending on signal speed).
- Select the active channels.
- To add protocol decoder (I2C, SPI, UART)
- Click the icon which is shown in the below picture.

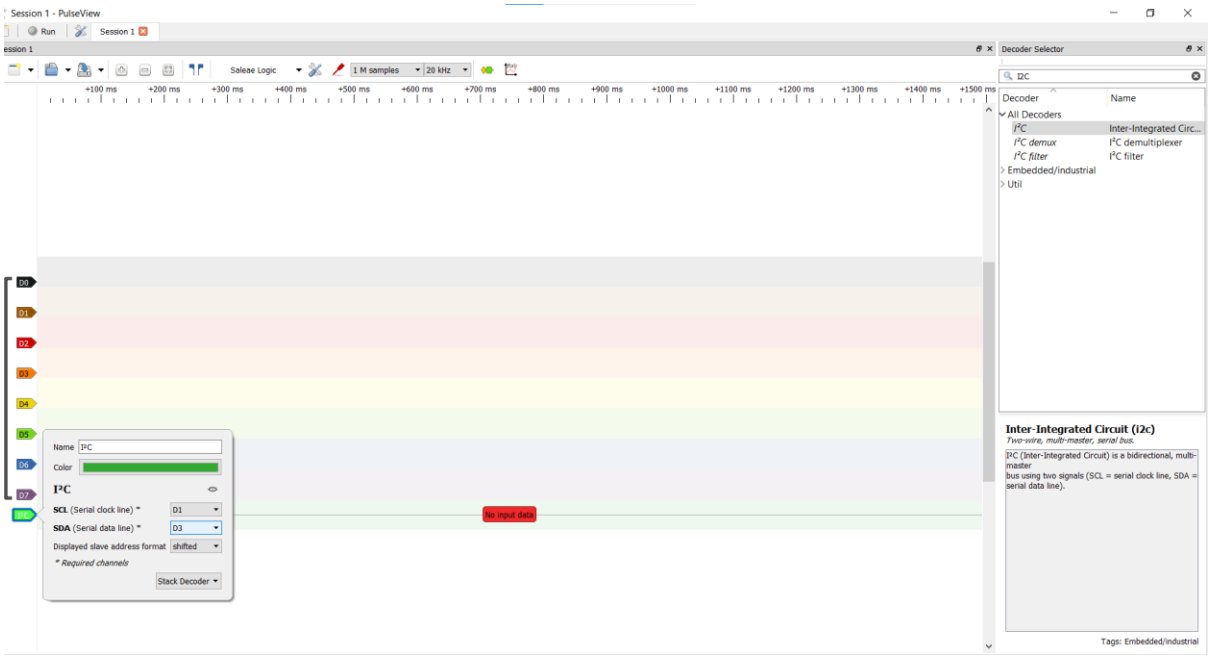


- You can search the communication protocols like I2C, SPI etc in search bar.

- Double Click the “I2C Communication protocol”

- In the left below corner, you can see an icon “I2C”

- Click that I2C and define the channels that you used for the I2C communication



- Make sure that common ground is connected.

Step 7: Start Capturing Signals

- Click the Run button.
- Operate your circuit (send data from MCU).
- Observe digital waveforms on the screen.
- Use zooms and measurement tools to analyse timing and data.

Examples to try Peripherals & Communication protocols

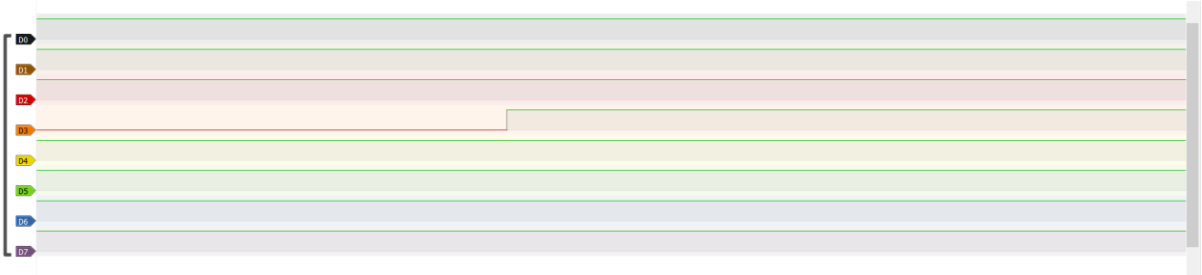
1) Digital Signal:

Description: The Digital signal has too Low for one second and High for one second for infinity times

Example Code:

```
// Basic ESP32-C6 Blink Code
#define GPIO = 12;
void setup() {
  pinMode(GPIO, OUTPUT);
}

void loop() {
  digitalWrite(GPIO, HIGH);
  delay(1000);
  digitalWrite(GPIO, LOW);
  delay(1000);
}
```

Note: It requires one GPIO Pin from MCU (ESP32C6) for one GPIO output



2) Pulse Width Modulation:

Description: The PWM Signal was generated between 0 to 100% (0 to 255 bits) of duty cycle with 500 Hz of frequency

Example Code:

```
// ESP32-C6 PWM Example
const int ledPin = 12;      // GPIO pin to use
const int freq = 5000;      // 5 kHz frequency
const int channel = 0;      // LEDC channel (0-15)
const int resolution = 8;   // 8-bit resolution (0-255)

void setup(){
  // Configure the PWM channel
```

```
  ledcAttachChannel(ledPin, freq, resolution, channel);
}

void loop(){
  // Increase brightness
  for(int dutyCycle = 0; dutyCycle <= 255; dutyCycle++){
    ledcWrite(ledPin, dutyCycle);
    delay(15);
  }
  // Decrease brightness
  for(int dutyCycle = 255; dutyCycle >= 0; dutyCycle--){
    ledcWrite(ledPin, dutyCycle);
    delay(15);
  }
}
```

Note: It requires one PWM Pin from MCU (ESP32C6) for one PWM output

3) Communication Protocols:

UART (Universal Asynchronous Receiver Transmitter)

Description: UART communication is configured with 8 data bits plus framing bits, resulting in a total of 10 bits per frame. The data transmission rate is 960 bytes per second, corresponding to a baud rate of 9600 Hz.

The UART communication is enabled on the MCU (ESP32-C6), which transmits the data value “20” (in decimal) to be received and read at the other end.

A single UART communication requires two MCU pins: one for Transmit (TX) and one for Receive (RX)

Example Code:

```
void setup() {
  Serial.begin(115200);          // USB serial for debug
  Serial1.begin(9600, SERIAL_8N1, 12, 13); // UART1: RX=16, TX=17

  while (!Serial); // wait for USB Serial
```

```
  Serial.println("UART Master Started");
}

void loop() {
  byte txData = 20;

  // Send 20 to slave
  Serial1.write(txData);
  Serial.print("Master Sent: ");
  Serial.println(txData);

  // Wait for ACK
  if (Serial1.available() > 0) {
    byte ack = Serial1.read();
    Serial.print("Master Received ACK: ");
    Serial.println(ack);
  }

  delay(1000); // 1 second delay
}
```

Note: Hexadecimal values are used for representation, while the controller processes data in binary.

4) I2C (Inter-Integrated circuit)

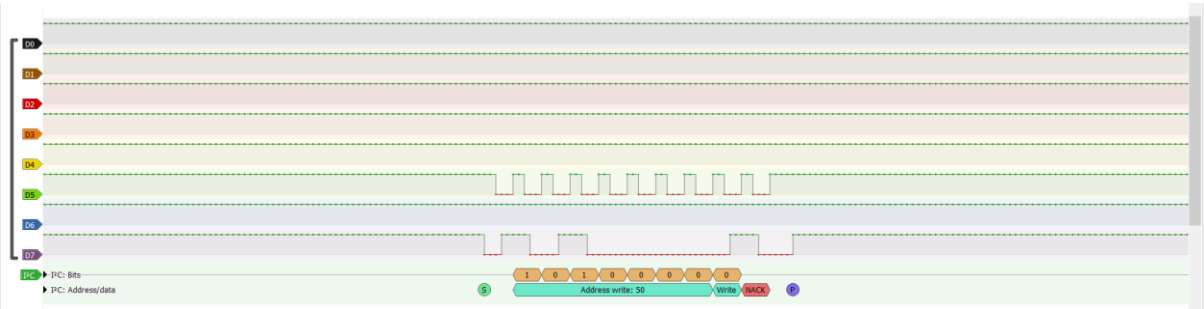
Description: I^2C communication is enabled with 8-bit data frames, where each byte transfer is followed by the required ACK bit, resulting in 9 bits per data transfer (excluding START and STOP conditions). The data transfer rate is approximately 960 bytes per second, corresponding to an I^2C clock frequency of 9.6 kHz.

The communication is initiated only when the MCU (ESP32-C6) generates the slave address on the bus. Data transmission proceeds after the addressed slave responds with an acknowledgment.

A single I2C communication requires two MCU pins: one for Serial data line (SDA) and one for Serial clock line(SCL).

Example Code:

```
#include <Wire.h>

#define SDA_PIN 12
#define SCL_PIN 13
#define I2C_FREQ 100000
#define SLAVE_ADDR 0x50

void setup() {
  Serial.begin(115200);
  Wire.begin(SDA_PIN, SCL_PIN);    // Master
  Wire.setClock(I2C_FREQ);

  Serial.println("I2C Master Started");
}

void loop() {
  uint8_t data[100];

  // Fill the array with values 0..99
  for (uint8_t i = 0; i < 100; i++) {
    data[i] = i;
  }

  Wire.beginTransmission(SLAVE_ADDR);
  Wire.write(data, 100);  // Send all 100 bytes
  Wire.endTransmission();

  Serial.println("Sent 100 bytes via I2C");
  delay(1000);
}
```



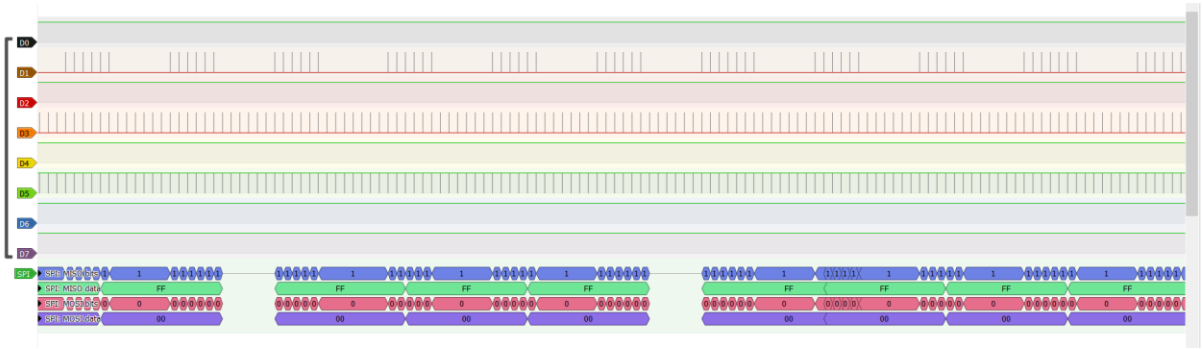
5) SPI (Serial Peripheral Interface)

Description: SPI communication is enabled with an 8-bit data frame, where data is transmitted synchronously with the clock signal. The SPI clock (SCK) operates at 1 MHz, resulting in a data rate of 1 Mbps (1 bit per clock cycle).

The MCU (ESP32-C6) acts as the SPI Master, generating the clock (SCK) and chip-select (CS) signals. Data is transmitted from the master to the slave through the MOSI line, while the MISO line is available for slave-to-master data transfer.

The SPI communication is enabled on the MCU (ESP32-C6), which transmits the data value “255” (in decimal) to be received and read at the other end.

Each SPI transaction occurs when CS is driven LOW, during which 8 clock pulses are generated to transfer one byte of data. SPI communication requires four signals from the MCU: SCK(serial clock), CS (Chip Select), MOSI(Master Out-Slave in), MISO (Master In – Slave out).



Note: Hexadecimal values are used for representation, while the controller processes data in binary.

Example Code:

```
#include <SPI.h>
/* SPI pin mapping */
#define MOSI_PIN 7
#define MISO_PIN 2   // not used, but required
#define SCK_PIN  6
#define CS_PIN   10
void setup() {
  pinMode(CS_PIN, OUTPUT);
  digitalWrite(CS_PIN, HIGH);
```

```
  // Initialize SPI with explicit pins
  SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, CS_PIN);

  // SPI settings: 8-bit, Mode 0
  SPI.beginTransaction(SPISettings(
    1000000,    // 1 MHz clock (good for logic analyzer)
    MSBFIRST,
    SPI_MODE0
  ));
}
void loop() {
  digitalWrite(CS_PIN, LOW);   // CS active
  SPI.transfer(0x7F);         // SEND ONLY 0x7F
  digitalWrite(CS_PIN, HIGH);  // CS inactive

  delay(1);                   // slow enough to see clearly
}
```